

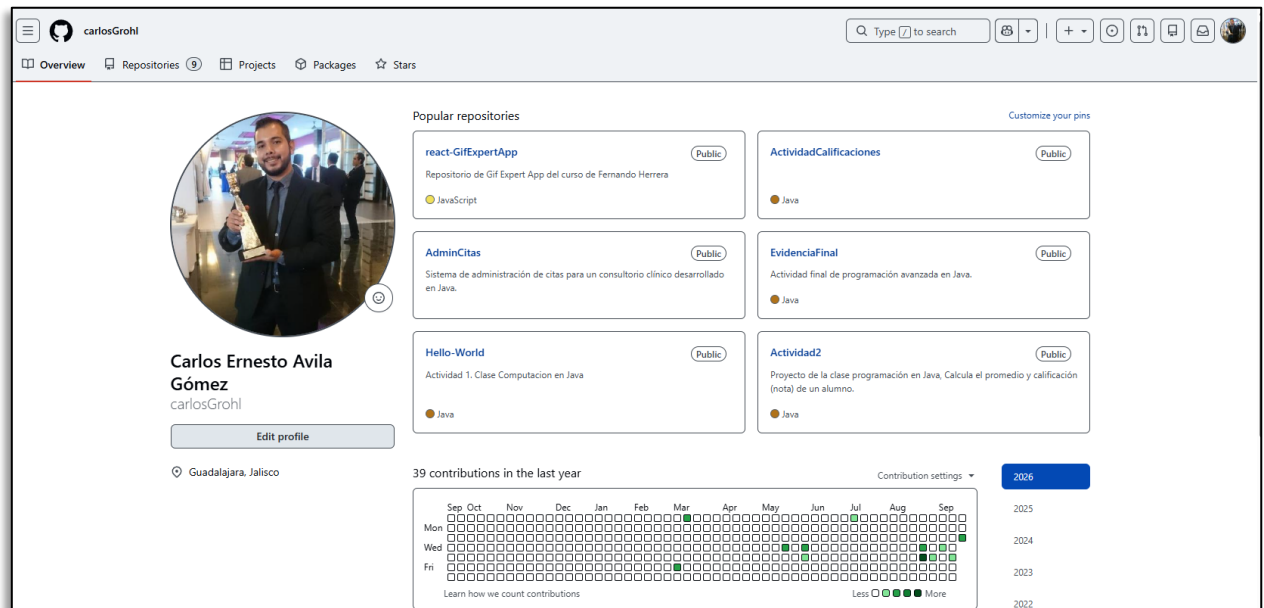
Portada.

Nombre: Carlos Ernesto Avila Gómez		Matrícula:7118681
Nombre del curso: computación en Java	Nombre del profesor: Silvia Tello Zuñiga	
Módulo: Semana 03.	Actividad: Evidencia 1 (Avance)	
Fecha: 22/09/2026		

Contenido

Portada.	1
Ambiente de Desarrollo.	2
Diagramas.	3
Diagrama de Flujo del programa principal.	4
Diagramas de Clases.	5
Pseudocódigo.....	8

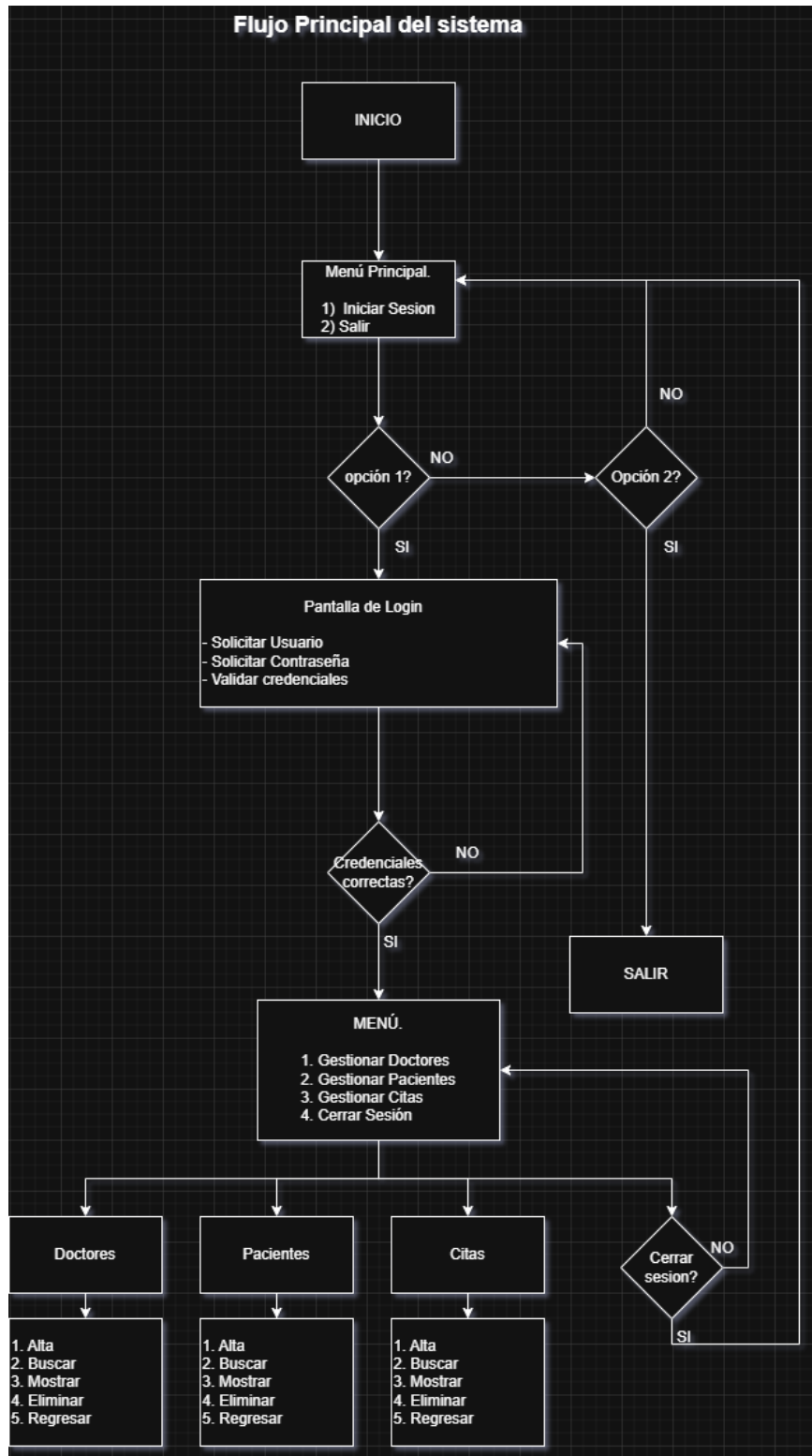
Evidencia de repositorio en GitHub.



Diagramas.

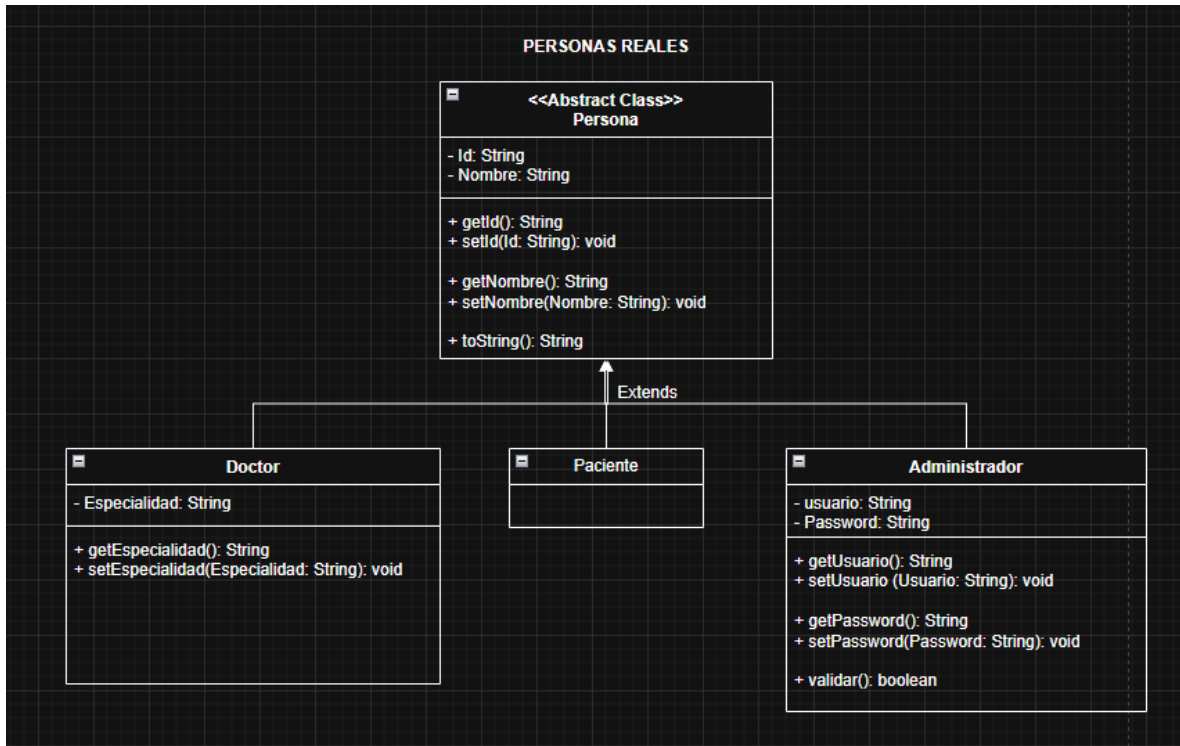
Para esta actividad he realizado los diagramas en la aplicación de Draw.IO, es una herramienta de Google que se puede usar en línea o instalada en Windows, tiene una interfaz muy amigable y permite realizar diagramas de diferentes tipos.

Diagrama de Flujo del programa principal.

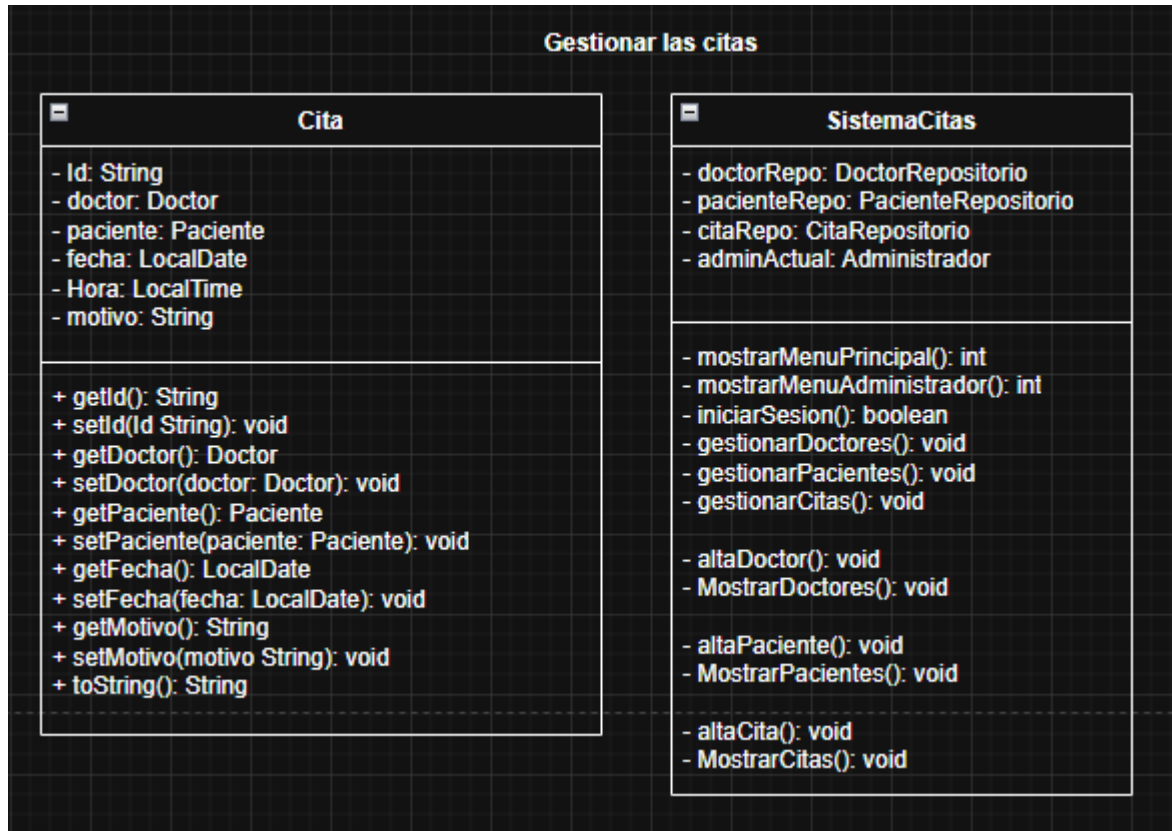


Diagramas de Clases.

Clases principales del programa.

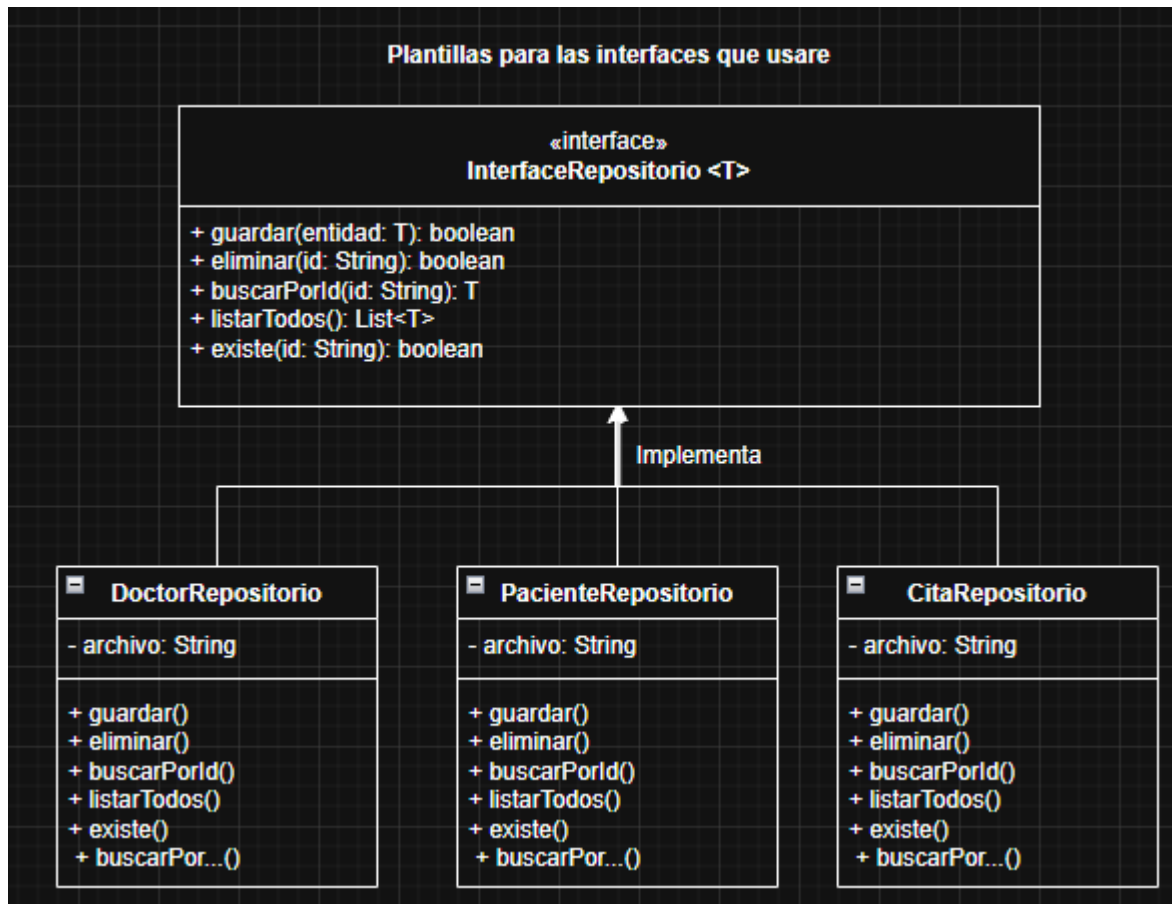


Clase cita y Clase para administrar las citas y el resto de las clases en general.



La administración de las 3 clases principales (Doctores, Pacientes y Citas) tienen demasiados métodos en común, pero no lo puedo abstraer como una entidad base de la cual vaya a crear herencias para cada una.

Por lo que, usare plantillas de la siguiente manera:



Pseudocódigo.

Clases principales.

Clase Persona

CLASE ABSTRACTA Persona

ATRIBUTOS:

- privado id: String
- privado nombre: String

CONSTRUCTOR:

```
Persona(id: String, nombre: String)
    this.id ← id
    this.nombre ← nombre
FIN CONSTRUCTOR
```

MÉTODOS:

```
getId(): String
    retorno id
FIN
```

```
setId(id: String): void
    this.id ← id
FIN
```

```
getNombre(): String
    retorno nombre
FIN
```

```
setNombre(nombre: String): void
    this.nombre ← nombre
método
```

```
toString(): String
    retorno id + " - " + nombre
FIN
```

FIN CLASE

CLASE Doctor EXTENDIENDO Persona**ATRIBUTOS:**

- privado especialidad: String

CONSTRUCTOR:

```
Doctor(id: String, nombre: String, especialidad: String)
    SUPER(id, nombre)
    this.especialidad ← especialidad
FIN
```

MÉTODOS:

```
getEspecialidad(): String
    retorno especialidad
FIN

setEspecialidad(especialidad: String): void
    this.especialidad ← especialidad
FIN

toString(): String
    retorno SUPER.toString() + " [" + especialidad + "]"
FIN
```

FIN CLASE**CLASE Paciente EXTENDIENDO Persona****CONSTRUCTOR:**

```
Paciente(id: String, nombre: String)
    SUPER(id, nombre)
FIN CONSTRUCTOR
```

MÉTODOS:

```
toString(): String
    retorno SUPER.toString()
FIN
```

FIN CLASE

CLASE Administrador**ATRIBUTOS:**

- privado usuario: String
- privado password: String

CONSTRUCTOR:

```
Administrador(usuario: String, password: String)
    this.usuario ← usuario
    this.password ← password
FIN CONSTRUCTOR
```

MÉTODOS:

```
getUsuario(): String
    retorno usuario
FIN
```

```
setUsuario(usuario: String): void
    este.usuario ← usuario
FIN
```

```
getPassword(): String
    retorno password
FIN
```

```
setPassword(password: String): void
    this.password ← password
FIN
```

```
estático validar(usuario: String, password: String): booleano
    archivo ← "data/administradores.csv"
    try
        lineas ← UtileriaArchivo.leerArchivo(archivo)
        para cada linea en lineas hacer
            partes ← linea.dividir(",")
            si partes[0] == usuario Y partes[1] == password entonces
                retorno VERDADERO
            fin si
        fin para
    catch Excepcion
        imprimir "Error al leer archivo de administradores"
    fin try
    retorno FALSO
FIN método
```

FIN CLASE

CLASE Cita**ATRIBUTOS:**

- privado id: String
- privado doctor: Doctor
- privado paciente: Paciente
- privado fecha: LocalDate
- privado hora: LocalTime
- privado motivo: String

CONSTRUCTOR:

```
Cita(id: String, doctor: Doctor, paciente: Paciente,  
    fecha: LocalDate, hora: LocalTime, motivo: String)  
    this.id ← id  
    this.doctor ← doctor  
    this.paciente ← paciente  
    this.fecha ← fecha  
    this.hora ← hora  
    this.motivo ← motivo
```

FIN CONSTRUCTOR**MÉTODOS:**

```
getId(): String  
    retorno id
```

FIN

```
setId(id: String): void  
    this.id ← id
```

FIN

```
getDoctor(): Doctor  
    retorno doctor
```

FIN

```
setDoctor(doctor: Doctor): void  
    this.doctor ← doctor
```

FIN

```
getPaciente(): Paciente  
    retorno paciente
```

FIN

```
setPaciente(paciente: Paciente): void
```

```
        this.paciente ← paciente
    FIN

    getFecha(): LocalDate
        retorno fecha
    FIN

    setFecha(fecha: LocalDate): void
        this.fecha ← fecha
    FIN

    getHora(): LocalTime
        retorno hora
    FIN

    setHora(hora: LocalTime): void
        this.hora ← hora
    FIN

    getMotivo(): String
        retorno motivo
    FIN

    setMotivo(motivo: String): void
        this.motivo ← motivo
    FIN

    toString(): String
        retorno id + " | " + fecha + " " + hora + " | " +
            doctor.getNombre() + " | " + paciente.getNombre()
    FIN
FIN CLASE
```